



QSOE

Quick and Secure Operating Environment

Programming Book

Writing native software for QSOE

Document: QSOE-PRG

Revision: Rev A — Draft

Applies to: QSOE 0.1

Yuri Zaporozhets

June 2026

© 2026 Yuri Zaporozhets.

Licensed under CC BY-ND 4.0 — verbatim redistribution with attribution; no derivatives.

<https://creativecommons.org/licenses/by-nd/4.0/>

Contents

1	Introduction	1
1.1	Who this book is for	1
1.2	What “native” programming means here	1
1.3	The shape of a QSOE program	2
1.4	This book and the others	2
1.5	What QSOE 0.1 is, and how this book treats it	3
1.6	How to read this book	3
2	The Programming Model	4
2.1	Processes: isolated by default	4
2.2	Threads: where concurrency lives	4
2.3	The two-thread shape	5
2.4	Creating a process: <code>posix_spawn</code>	5
2.5	Process lifetime and exit	6
3	Message Passing	7
3.1	Two endpoints: channels and connections	7
3.2	Finding a server: the namespace	7
3.3	The rendezvous: Send, Receive, Reply	8
3.3.1	The reply contract	8
3.4	What the server learns: <code>_msg_info</code>	9
3.5	Scatter/gather: vectored messages	9
3.6	Deferring a reply	10
3.7	Large messages	10
3.8	A worked example: a counter server	11
4	Pulses and Signals	13
4.1	What a pulse is	13
4.2	Sending and receiving pulses	13
4.3	Signals are pulses	14
4.4	The POSIX signal surface	15
4.5	Pulses as notification: the <code>sigevent</code>	15
5	Synchronization	17
5.1	Two faces, one mechanism	17
5.2	The primitives	17
5.3	No futex: the address is the identity	18
5.4	Priority inheritance: the real-time face	18
5.5	Synchronizing across processes	19
5.6	Choosing a tool	19

6	Time and Timers	20
6.1	Clocks: reading the time	20
6.2	Sleeping: blocking for a duration	20
6.3	Asynchronous and periodic timers	21
6.4	Timed blocking	21
6.5	Time and real time	22
7	Interrupt Threads	23
7.1	No handlers, only threads	23
7.2	The interrupt API	23
7.3	The canonical IST loop	24
7.4	The IST and the rest of the program	24
7.5	One source per thread	25
8	The Resource-Server Framework	26
9	A Complete Resource Server	27
A	Native API Quick Reference	28

Chapter 1

Introduction

This book is for someone writing *new* software for QSOE: a resource server, a device driver, a program built around QSOE’s own message passing rather than ported from another system. It teaches the native QSOE programming model from the ground up — how processes and threads are arranged, how they talk to one another, and how the system services that hang off that conversation (synchronization, time, interrupts, and the resource-server framework) are built.

1.1 Who this book is for

You should read this book if you are building *with* what QSOE provides. If instead you have an existing Unix, POSIX, or QNX program and want it to run on QSOE, the *Application Porting Guide* (QSOE-APG) is the better starting point: it is about reconciling outside code with QSOE, where this book is about writing code that is QSOE-shaped from the first line. The two meet in the middle — a substantial port often grows a native resource server, and a native program still uses the standard C library — but they start from opposite ends.

A working knowledge of C and of the general POSIX/Unix programming model is assumed. Familiarity with QNX message passing is helpful, because QSOE’s model is openly inspired by it, but it is not required: this book builds the model up without assuming you already know the QNX one.

1.2 What “native” programming means here

A QSOE program is not merely a POSIX program that happens to run on QSOE. The standard C library is there, and a great deal of ordinary C code works unchanged — but the system’s real shape is a **message-passing microkernel**, and the interesting software (drivers, servers, anything that offers a service to other programs) is written directly to that shape. “Native” programming, in this book, means working with the primitives the system is actually built from:

- **Channels and connections** — the endpoints a program creates to receive messages, and the links other programs open to it.
- **Message passing** — the `MsgSend/MsgReceive/MsgReply` rendezvous that is the one way QSOE programs exchange requests and replies, and the substrate almost everything else is built on.
- **Pulses** — short, fixed, non-blocking messages used for asynchronous notification, including how signals are delivered.
- **The service APIs** — synchronization (`Sync*`), time and timers (`Timer*`), and interrupt threads (`InterruptAttachThread`) — each of which, underneath, is a conversation with the system.

- **The resource-server framework** — the structured way to offer a path in the namespace and answer POSIX I/O on it, so that clients reach your code through ordinary `open/read/write`.

Inspired by QNX, not compatible with it. QSOE's API is *QNX-inspired*, deliberately, but it is not QNX-compatible and does not try to be. There is no QNX/RISC-V binary interface to preserve, so message layouts, the exact call set, pulse formats, and the resource-server framework are all free design choices — and several differ from QNX on purpose, usually to be simpler. The call set is not even the same size in both directions: QSOE adds primitives QNX has no need for. `MsgSaverreply` is the clearest example — a server calls it to capture its reply context before parking a client to answer later (a driver waiting on input, a pipe with an empty buffer). QNX needs no such call, because there a received message's reply handle stays valid on its own until replied; QSOE's is implicit and must be saved deliberately before the next receive. Where this book draws a QNX comparison it is to orient a reader who knows that system, never a promise of source compatibility. The one place QSOE *does* aim at QNX compatibility is the standard C library (the 1.0 goal of QSOE-APG); the native API described in this book is its own thing.

1.3 The shape of a QSOE program

Two facts about every QSOE program are worth stating at the outset, because they color everything that follows.

Every program runs two threads from the start. From the moment it begins, a QSOE program has not one thread but two: your **main thread**, which runs `main()`, and a small **system thread** that the C runtime starts for you. The system thread receives the pulses the system sends your process — most visibly, it is where signal handlers run (Chapter 4). You do not start it and usually do not see it, but it is always there, and a few rules in this book (notably around signals and interrupt threads) only make sense once you know it exists. This is a permanent part of the model, not an implementation detail that might change.

New processes come from `posix_spawn`, only. QSOE has no `fork()`: a new process is created in one step from a program on a path, with `posix_spawn()`, never by cloning the caller. This book takes that as given and does not re-argue it — the *Application Porting Guide* treats the consequences for outside code, and the *Design & Architecture* guide explains the reasoning — but it shapes how native programs are structured: concurrency within a process is built from threads, and separate processes cooperate by message passing, never by sharing an inherited address space.

C, not C++. QSOE is a C-only environment: there is no C++ runtime or standard library, and the headers carry no `extern "C"` guards. Every example in this book is C, and every API it describes is a C API.

1.4 This book and the others

QSOE's documentation is four manuals, and this one assumes you can reach the other three:

- *Design & Architecture* (QSOE-DSN) — *why* the system is built as it is: the two-kernel model, the boot chain, the IPC rationale. This book points to it for reasoning it does not repeat.
- *User Guide* (QSOE-USR) — installing, booting, and operating a QSOE system, which you will want running to try anything here.

- *Application Porting Guide* (QSOE-APG) — bringing existing software across; the companion to this book from the other direction.
- *Programming Book* (QSOE-PRG) — this manual.

Where a topic genuinely belongs to another manual — the design reasoning behind `no-fork`, the full inventory of missing `libc` functions — this book refers you there rather than duplicating it, so that the four stay consistent as the system grows.

1.5 What QSOE 0.1 is, and how this book treats it

This book describes QSOE **0.1**. The native programming model — the message passing, the two-thread shape, the resource-server design — is settled and is what you should write to. But 0.1 is an early release: some of the surface described here is fully working and exercised by the system’s own drivers and shell, while some is designed and specified but still being implemented.

This book is honest about the difference. Where a facility is the settled, working API, it is described plainly. Where a facility is designed but not yet complete in 0.1 — the resource-server framework most notably, whose object model is fixed but whose names are still provisional and whose implementation is in progress — it is marked clearly as planned, so you can tell what you can build on today from what you are reading as a forward design. Nothing here is presented as shipping that is not.

1.6 How to read this book

- Chapter 2 — the programming model: processes, threads, the two-thread shape, and how a native program is structured.
- Chapter 3 — message passing: channels and connections, the `MsgSend/MsgReceive/MsgReply` rendezvous, and the reply contract. The core of the book.
- Chapter 4 — pulses and signals: asynchronous notification, and how signals reach your handlers.
- Chapter 5 — synchronization: the `Sync*` API and the `pthread`s layer built on it.
- Chapter 6 — time and timers.
- Chapter 7 — interrupt threads: servicing hardware from an ordinary thread, with no in-process handler.
- Chapter 8 — the resource-server framework: offering a path and answering POSIX I/O on it.
- Chapter 9 — a complete resource server, end to end, tying the previous chapters together.
- Appendix A — a quick reference to the native API.

The chapters build on one another, especially Chapters 3 through 7, which are best read in order the first time. If you have one goal — to write a resource server — you can read Chapters 2 and 3, then jump to Chapter 8 and use the rest as reference.

Chapter 2

The Programming Model

Before two programs can exchange a message, there have to *be* programs — arranged as processes and threads, created and torn down in particular ways. This chapter lays out that structural model: what a process is on QSOE, how threads live inside it, the two-thread shape every program carries, how a new process comes into being, and how one ends. None of it is message passing yet — that is the next chapter — but all of it is the ground message passing stands on.

2.1 Processes: isolated by default

A QSOE process is a running program with its own private address space, identified by a **pid**. Two processes share *nothing* of their memory unless they have deliberately arranged to — there is no implicit sharing of any kind. Each new process is built from fresh memory frames; QSOE does no copy-on-write (the absence of **fork** leaves nothing for it to optimize, so it is a hard non-goal). The *Application Porting Guide* and the *Design & Architecture* guide treat the reasoning; for writing native software, the consequence is the thing to absorb:

Isolation is the default. Sharing — of memory, of data, of work — is always something a program asks for explicitly, never something it inherits.

This single principle shapes the whole programming model. Two processes that must cooperate do so by **message passing** (Chapter 3), or, where they genuinely need shared memory, by an explicit shared-memory arrangement — never by the accidental sharing a **fork**-based system hands out. When you want concurrency *within* a single address space, that is what threads are for.

2.2 Threads: where concurrency lives

The threads of one process share that process’s address space: the same code, the same globals, the same heap. Concurrency inside a program is built from threads; cooperation between programs is built from messages. Keeping those two straight is most of what “thinking in QSOE” amounts to.

Two ways to start a thread. QSOE offers both a portable and a native spelling, over the same underlying mechanism:

- **pthread_create** — the POSIX interface, for portable code and for anything that also has to build elsewhere.

- **ThreadCreate** — the native, QNX-shaped call: `ThreadCreate(pid, func, arg, attr)`. A `pid` of 0 means “this process”; QSOE creates threads only in the calling process, so that is the only value 0.1 accepts. It returns the new thread id (`tid`, always ≥ 2 , since the main and system threads take the first slots).

Most code should use `pthread_create`. Reach for `ThreadCreate` when you want the native attribute struct directly — to set a thread’s priority or its SMP run-mask at creation — or when you are reading QSOE’s own servers, which use it. The companion calls follow the same two faces: `ThreadJoin`/`pthread_join`, `ThreadDetach`, and so on.

Scheduling is priority-driven and preemptive. Every thread has a priority in the range **1 to 255** (higher is more urgent); a thread that specifies none gets the default, 254. Scheduling is strict-priority *preemptive*, not cooperative: a runnable higher-priority thread displaces a lower one immediately, on whichever processor. The standard policies `SCHED_FIFO`, `SCHED_RR`, and `SCHED_OTHER` are available, and a thread’s policy and priority can be read and set at runtime with `SchedGet` and `SchedSet`. QSOE is SMP from the start and built for hard real time; the guarantees that follow from that — bounded preemption, real-time thread classes — are the subject of the *Design & Architecture* guide and are not re-derived here.

2.3 The two-thread shape

Here is the fact that most distinguishes a QSOE process from a textbook Unix one: **every process runs two threads from the very start**, not one.

- The **main thread** runs your `main()`, exactly as you expect.
- The **system thread** is started for you by the C runtime, *before* `main()` begins. You do not create it and usually never see it.

The system thread exists to receive the short asynchronous messages — pulses (Chapter 4) — that the system sends your process. Its most visible job is **signal delivery**: it owns a dedicated channel that the C library registers with the system server as this process’s signal channel, and `kill` reaches your process by sending a pulse there. The system thread decodes each such pulse and runs the matching signal handler.

That placement has a consequence worth stating now, because it explains the model: **your signal handlers run on the system thread**, not by interrupting the main thread’s stack mid-instruction. Delivery is deferred to a real thread context rather than injected asynchronously, which removes a whole class of async-signal-safety hazards — but the handler still shares the address space with the main thread, so ordinary data shared between them needs the ordinary care (Chapter 5).

You will not manage the system thread, but a few things in this book only make sense once you know it is there: it is why a signal sent from another process can run your handler at all, and it is the reason a clean process exit tears down two threads, not one (§2.5). This two-thread shape is a permanent part of the QSOE model, the same on both kernels.

2.4 Creating a process: `posix_spawn`

A new process is created in exactly one way: from a program named by a path, in a single step. The portable spelling is `posix_spawn` (and the PATH-searching `posix_spawnnp`); the native call underneath is `ProcessCreate`, which takes a path and returns a pid. There is no `fork`, no `exec`: `posix_spawn` is the create-and-run step as one call.

```
pid_t pid;
char *const argv[] = { "ls", "-l", NULL };
int rc = posix_spawn(&pid, "/bin/ls", NULL, NULL, argv, environ);
if (rc != 0) { /* rc is the errno */ }
/* parent continues; later: waitpid(pid, &status, 0); */
```

The child is a fresh, independent program loaded from the path — never a copy of the caller. It receives the `argv` and `envp` you pass, and it starts at *its* `main`, not at the point of the spawn.

File-descriptor inheritance. In QSOE 0.1 a spawned child automatically inherits the parent’s open file descriptors that are *not* marked close-on-exec — `stdin`, `stdout`, and `stderr` among them. A freshly spawned program therefore wakes with its standard streams already connected to wherever the parent’s were, with no work on either side. To withhold a descriptor from a child, set `FD_CLOEXEC` on it before spawning; it is then left out of the snapshot.

Planned content. The `file_actions` and `attr` arguments to `posix_spawn` are accepted by the API but **not yet honored in 0.1**. The per-child descriptor surgery a `file_actions` list expresses — redirecting one child’s `stdout` into a pipe, closing a specific fd for that child alone — is planned but not available; today inheritance is all-or-by-`CLOEXEC` as described above. Likewise the `posix_spawnattr_t` controls (a child’s initial signal mask, scheduling policy, and the like) are not yet applied. Pass `NULL` for both in 0.1; code that needs per-child redirection should wait for these or factor the work into a small helper program that arranges its own descriptors after it starts.

2.5 Process lifetime and exit

A process runs until it ends, which happens in one of three ways:

- It **returns from main** or calls `exit` / `_exit` — the ordinary voluntary exit, carrying an exit status.
- It is **terminated** — by `ProcessTerminate`, or by a signal whose default action is to end the process.
- It **faults** — a stray pointer or other fatal trap. Rather than panicking the kernel, QSOE turns a U-mode fault into a clean terminated-by-signal exit (the C library registers a crash landing pad at start-up), so a buggy program brings down only itself and reports a signal the same way a Unix core-dump would.

A parent collects a child’s outcome with `waitpid` (or `wait`), and reads it with the usual `WIFEXITED` / `WEXITSTATUS` and `WIFSIGNALED` / `WTERMSIG` macros — a voluntary exit reports its status, a terminated one reports the signal. Once waited for, a finished process leaves nothing behind.

The two-thread shape closes the loop here. On a clean exit the C library puts the system thread away — it is parked in `MsgReceive` between pulses — before the final teardown, so the kernel’s last-thread cleanup fires on the main thread as it leaves. You never manage this; it is named only so the symmetry is visible: a process that started as two threads ends by retiring both, in order.

Chapter 3

Message Passing

This is the chapter the rest of the book leans on. Message passing is not one QSOE facility among many — it is *the* mechanism by which separate programs interact at all. Opening a file, reading a device, asking the system server to spawn a process, talking to a driver you wrote yourself: underneath, every one of them is a message sent to a server and a reply sent back. Learn this chapter and the resource-server framework of Chapter 8 becomes a convenience over machinery you already understand.

Message passing here is **synchronous**: a client sends and blocks until the server replies. The short, non-blocking cousin — pulses — is the subject of Chapter 4; this chapter is the synchronous rendezvous.

3.1 Two endpoints: channels and connections

The conversation has two asymmetric ends:

- A **channel** is a server's receiving end. A server creates one with `ChannelCreate`, which returns a **chid** (channel id), and then receives messages on it. `ChannelDestroy` closes it.
- A **connection** is a client's sending end. A client opens one to a server's channel with `ConnectAttach`, which returns a **coid** (connection id), and then sends messages over it. `ConnectDetach` closes it.

Many connections may attach to one channel: a single server channel fields requests from every client that has connected to it. A chid is meaningful to the server that owns it; a coid is meaningful to the client that holds it. The two are not interchangeable, and neither is a global address — they are local handles on opposite sides of one link.

3.2 Finding a server: the namespace

`ConnectAttach` needs to name the server's channel:

```
int ConnectAttach(uint32_t nd, pid_t pid, int chid, unsigned index,
                  int flags);
```

so a client must somehow learn the server's `pid` and `chid`. It learns them from the **namespace**. A server publishes its channel under a path, and a client resolves that path back to the coordinates:

```
/* server side: announce that open("/dev/thing") reaches my channel */
qsoe_pathmgr_register("/dev/thing", chid);

/* client side: resolve the path, then attach */
```

```
pid_t pid; int chid; unsigned kind;
qsoe_pathmgr_resolve("/dev/thing", &pid, &chid, &kind);
int coid = ConnectAttach(0, pid, chid, 0, 0);
```

The path manager lives inside the system server (**taskman**), and the single most important property of this arrangement is what happens *after* the resolve: **taskman is in the lookup, never in the data path**. Once the client holds a coid, its messages travel straight to the server and the replies straight back; the system server is not a relay and never sees the traffic. This is why message passing can be fast enough to build a whole system on.

In practice you will rarely call `qsoe_pathmgr_resolve` yourself. An ordinary client says `open("/dev/thing")`, and the C library and the resource-server framework (Chapter 8) perform exactly the resolve-and-attach above on your behalf, handing you back a file descriptor that *is* a connection. The primitives are shown here so that when you write the server end, you recognize the machinery underneath `open`.

3.3 The rendezvous: Send, Receive, Reply

Three calls carry the synchronous exchange:

```
int MsgSend    (int coid, const void *smsg, int sbytes,
                void *rmsg, int rbytes);
int MsgReceive(int chid, void *msg, int bytes, struct _msg_info *info);
int MsgReply   (int rcvid, int status, const void *msg, int bytes);
```

The flow, and the thread states it moves through, are the core of the model:

1. The client calls `MsgSend`, handing over `sbytes` from `smsg`. It now **blocks** — first SEND-blocked until the server receives the message, then REPLY-blocked until the server replies.
2. The server, sitting in `MsgReceive`, wakes with the message in its buffer and a **rcvid** (receive id) identifying this particular request. It does its work.
3. The server calls `MsgReply` with that `rcvid`, a **status**, and up to `bytes` of reply payload. The client's `MsgSend` returns and it runs again, with the reply now in `rmsg`.

A server with nothing to do is RECEIVE-blocked in `MsgReceive`; a client awaiting an answer is REPLY-blocked in `MsgSend`. No buffering, no queue in the middle: the two threads meet, exchange, and part.

3.3.1 The reply contract

One detail of these signatures is worth dwelling on, because it is where QSOE is cleaner than the system it was inspired by. **The reply status is a separate value from the reply payload:**

- `MsgReply`'s **status** argument is the server's result code.
- That same value is what the client's `MsgSend` *returns*.
- The reply *message* — the `msg/bytes` of `MsgReply`, landing in the `rmsg` of `MsgSend` — is **pure payload**, carrying no status word of its own.

So a client reads its result in the most natural way imaginable: the return value of the send *is* the status, and the buffer holds only data.

```
int status = MsgSend(coid, &req, sizeof req, &reply, sizeof reply);
if (status < 0) { /* transport failure: qsoe_errno is set */ }
else             { /* status is the server's result; reply holds the data */ }
```

A negative return means the exchange itself failed (the server vanished mid-request, the connection was bad) and `qsoe_errno` is set. A zero-or-positive return is the status the server chose, with 0 conventionally meaning success. An application never assembles or parses a raw message buffer with a status field baked into it; status travels in its own channel, data in its own.

3.4 What the server learns: `_msg_info`

`MsgReceive`'s last argument is filled in with a description of the message and its sender:

```
struct _msg_info {
    uint32_t nd;           /* sending node (local for now) */
    pid_t pid;            /* sender's pid */
    int chid;             /* channel it arrived on */
    int scoid;            /* server-side connection id */
    int msglen;           /* bytes received (clamped to buffer) */
    int srcmsglen;        /* bytes the sender actually sent */
    int priority;         /* sender's scheduling priority */
    int flags;            /* QSOE_MI_PULSE if this was a pulse */
    /* ... */
};
```

Two fields earn early mention. `srcmsglen` versus `msglen` lets a server detect a message larger than the buffer it offered, rather than silently truncating. And `flags` carries `QSOE_MI_PULSE`: the same `MsgReceive` returns both real messages and pulses, so a server that can receive pulses must test this bit before treating the buffer as a request (Chapter 4). For richer client identity — and, later, credentials — a server can call `ConnectClientInfo` with the `scoid`.

Priority inheritance. When a high-priority client sends to a server, the server's receiving thread runs at the *client's* priority while handling that request. A low-priority client cannot, by holding a server busy, make a high-priority client wait behind it — the classic priority inversion — and a real-time client's urgency follows its request into the server. This is automatic; you do not arrange it.

3.5 Scatter/gather: vectored messages

Often a message is naturally in pieces — a fixed header followed by a variable body — and copying them into one contiguous buffer first is wasted work. The vectored calls send and receive directly from several buffers, described by an array of `iov_t` (QSOE's name for the POSIX `struct iovec`):

```
iov_t siov[2];
SETIOV(&siov[0], &header, sizeof header); /* part 1: the header */
SETIOV(&siov[1], body, body_len);          /* part 2: the payload */
MsgSendv(coid, siov, 2, &riov, 1);         /* sent as one message */
```

`MsgSendv`, `MsgReceivev`, and `MsgReplyv` mirror the three scalar calls, taking an IOV array and a part count in place of a single buffer/length. The receiver sees one contiguous message regardless of how many parts the sender split it into; the split is purely a convenience on each side. This is the everyday shape of a real request: a typed header part that says what is wanted, and a payload part that carries the data.

3.6 Deferring a reply

A server need not reply before it receives again — but there is a rule it must respect to do so. **The reply context a `MsgReceive` hands you is implicit and single-slot: the next `MsgReceive` on that thread overwrites it.** Reply to each message before receiving the next, or capture the context first.

Capturing it is what `MsgSavereply` is for. A server that cannot answer yet — a device with no data ready, a pipe with an empty buffer — saves the reply context, returns to receiving other clients, and replies later when the answer arrives:

```
int saved = MsgSavereply(rcvid); /* park this client */
if (saved == -1) { /* EAGAIN: save-slot pool exhausted */ }
/* ... stash 'saved', go back to MsgReceive; meanwhile other clients run ... */
/* ... later, when data arrives: */
MsgReply(saved, 0, &data, len); /* now answer the parked client */
```

There is one sharp edge, called out because it has bitten real code: a *successful* `MsgSavereply` returns a **negative** integer (the saved rcvid has its top bit, `QSOE_RCVID_SAVED`, set). Test for failure with `saved == -1`, **never** with `saved < 0` — the latter would treat every successful park as an error.

This is the manual form of a pattern the resource-server framework (Chapter 8) wraps up as a single `qsoe_defer()` call. When you write a server by hand, `MsgSavereply` is how you park a client; when you write one on the framework, the framework parks it for you.

3.7 Large messages

Nothing special is required to send a big message. A payload larger than the in-line IPC buffer is transferred directly between the two address spaces by the kernel, in pieces, transparently — the same `MsgSend/MsgReply` move multi-megabyte buffers (up to a generous cap) exactly as they move a few words. A server that would rather not hold an entire large request in memory at once can read it out of the sender piecemeal with `MsgReadv`, and write the reply back piecemeal with `MsgWritev`, addressing the sender’s buffer by offset without ever buffering the whole thing. For most code, though, a large message is just a large `sbytes` — the mechanism scales without a new API.

And it does not cost you interrupt latency. This is the part that matters in a hard-real-time system. A multi-megabyte copy inside the kernel could, done naively, be a latency catastrophe: one long uninterruptible `memcpy` would lock out every higher-priority thread and stall interrupt servicing for as long as it ran. QSOE does not do it that way. The bulk transfer is broken into page-sized chunks with a **scheduling preemption point at each one**, so a higher-priority thread — an interrupt-service thread above all (Chapter 7) — can displace the copy at a page boundary instead of waiting out the whole message. A large message therefore costs *throughput*, not *bounded latency*: it never becomes a stretch during which the system stops answering its interrupts. The per-kernel mechanism and the exact latency bound are the *Design & Architecture* guide’s territory; what a programmer relies on here is the guarantee — message size does not trade against real-time responsiveness.

If a transfer fails partway. The wider window of a large copy makes a fair question concrete: what if something goes wrong in the middle of it — a thread ends, a process faults — while bytes are still moving? The reassuring answer is that message size changes nothing here either: a synchronous exchange always reaches a *definite* end, on a sixteen-megabyte message exactly as on a one-word one. Two guarantees cover every case:

- **The sender is frozen for the whole transfer.** A client in `MsgSend` is `REPLY`-blocked from the moment it sends until the reply lands: it cannot run, cannot return, and cannot free or alter the buffers the server is copying. So the copy never races the client mutating its own memory, and the sender's thread cannot quietly cease to exist underneath the copy — it is blocked, by construction, until the rendezvous completes, and its pages stay pinned for the duration.
- **Every failure resolves the rendezvous; none strands it.** If the server faults or exits partway through — even mid-copy — the kernel *completes* the waiting `MsgSend` rather than leaving it blocked forever: the call returns a negative error (`ESRVRFAULT`), with no reply data. A copy that faults on a bad page is turned into that same send status. The outcomes that *cannot* happen are an indefinite hang and a silently truncated reply mistaken for success.

So the client always lands in one of two places: it gets its reply and the server's status, or it gets a clean error. “Stuck halfway” is not a state the model allows.

3.8 A worked example: a counter server

Putting the pieces together — channel, registration, receive loop, reply — a complete server is short. This one hands out a monotonically increasing count, one tick per request:

```
#include <stdio.h>
#include <sys/qsoe.h>

#define COUNTER_PATH "/dev/counter"

int main(void)
{
    int chid = ChannelCreate(0);
    if (chid < 0) { perror("ChannelCreate"); return 1; }
    if (qsoe_pathmgr_register(COUNTER_PATH, chid) != 0) {
        perror("pathmgr_register"); return 1;
    }

    long count = 0;
    for (;;) {
        long req;
        struct _msg_info info;
        int rcvid = MsgReceive(chid, &req, sizeof req, &info);
        if (rcvid < 0) continue;           /* receive error: keep serving */
        if (info.flags & QSOE_MI_PULSE) continue; /* not a request (ch. 4) */

        long reply = ++count;             /* one tick per request */
        MsgReply(rcvid, 0, &reply, sizeof reply);
    }
}
```

and the client that drives it resolves the path, attaches, sends one request, and reads the count out of the reply:

```
#include <stdio.h>
#include <sys/qsoe.h>

#define COUNTER_PATH "/dev/counter"

int main(void)
```

```
{
    pid_t pid; int chid; unsigned kind;
    if (qsoe_pathmgr_resolve(COUNTER_PATH, &pid, &chid, &kind) != 0) {
        perror("resolve"); return 1;
    }
    int coid = ConnectAttach(0, pid, chid, 0, 0);
    if (coid < 0) { perror("ConnectAttach"); return 1; }

    long req = 0, reply = 0;
    int status = MsgSend(coid, &req, sizeof req, &reply, sizeof reply);
    if (status < 0) { perror("MsgSend"); return 1; }

    printf("counter = %ld\n", reply);          /* status 0 = EOK; data in reply */
    ConnectDetach(coid);
    return 0;
}
```

Every element of this chapter is visible in those two programs: the server's channel and its registration, the client's resolve-and-attach, the send/receive/reply rendezvous, the status-as-return-value contract, and the pulse check that keeps the receive loop honest. The rest of the book builds outward from exactly this shape — pulses add asynchronous notification to the receive loop (Chapter 4), and the resource-server framework (Chapter 8) wraps the loop so clients can reach the server with plain `open` and `read` instead of the raw calls shown here.

Chapter 4

Pulses and Signals

Chapter 3 was the synchronous half of message passing: a client sends, blocks, and waits for a reply. This chapter is the asynchronous half. A **pulse** is a short, fixed-size message that carries no reply and blocks no one — the way one program tells another that something *happened*, rather than asking it to do something and waiting. And then the payoff that ties the two chapters together: **on QSOE, signals are pulses**. The QSOE_MI_PULSE check that kept the counter server’s receive loop honest in §3.8 is about to earn its keep.

4.1 What a pulse is

A pulse is a small fixed structure, not an arbitrary buffer:

```
struct _pulse {
    uint16_t type;          /* _PULSE_TYPE (0) for application pulses */
    uint16_t subtype;
    int8_t code;           /* user-defined, signed 8-bit          */
    uint8_t reserved[3];
    union {
        int32_t sival_int;
        void *sival_ptr;
    } value;               /* user-defined, 32-bit          */
    int32_t scoid;         /* the sender connection, server’s view */
};
```

Everything that makes a pulse different from a message follows from one fact: **a pulse expects no reply**. The sender does not block — it fires the pulse and moves on, the way `kill` returns the instant a signal is *generated*, not when it is handled. There is no rendezvous, no REPLY-blocked state, nothing to `MsgReply` to. The whole payload is the `code` (a signed byte you assign meaning to) and the `value` (a 32-bit integer or pointer); a pulse says “this happened, here is one number about it,” and nothing larger.

That narrowness is the point. A pulse is for *notification* — a timer expired, data is ready, a child changed state — in exactly the situations where a synchronous request would be wrong: where the notifier must not block, or runs somewhere it cannot afford to wait. When you need to hand over real data, send a message; when you need only to say “something happened,” send a pulse.

4.2 Sending and receiving pulses

A pulse travels over a connection, just as a message does, so the sender first holds a coid to the target’s channel (§3.2). Sending is one call:

```
int MsgSendPulse(int coid, int priority, int code, int value);
```

The `priority` sets the pulse’s delivery priority; passing `SIGEV_PULSE_PRIO_INHERIT (-1)` tells it to inherit the receiver’s instead of pinning a fixed one. `code` and `value` are yours to define. Unlike `MsgSend`, this call does not block waiting for an answer — there will not be one.

Receiving needs no new primitive. The same `MsgReceive` a server already runs returns pulses *and* messages; the `QSOE_MI_PULSE` bit in `_msg_info.flags` tells them apart, and for a pulse the buffer holds a `_pulse`. A complete server loop therefore branches once:

```
for (;;) {
    union { struct _pulse pulse; my_request_t req; } m;
    struct _msg_info info;
    int rcvid = MsgReceive(chid, &m, sizeof m, &info);
    if (rcvid < 0) continue;

    if (info.flags & QSOE_MI_PULSE) {
        handle_pulse(&m.pulse);           /* react; do NOT reply */
    } else {
        MsgReply(rcvid, do_work(&m.req), &result, sizeof result);
    }
}
```

The one rule is in the comment: you never `MsgReply` to a pulse. There is no parked sender behind it and no reply context to consume — a pulse is already complete the moment it arrives. (A server that wants *only* pulses, never messages, can use `MsgReceivePulsev` instead, but the unified loop above is the common shape.)

4.3 Signals are pulses

Here is where the chapter pays off. QSOE has no separate kernel signal machinery: **a POSIX signal is delivered as a pulse**, through the very mechanism just described. Recall from §2.3 that every process runs a system thread, and that this thread owns a dedicated channel which the C library registers with the system server as the process’s *signal channel*. That channel is where signals arrive.

Follow a `kill` across the system and there is nothing in it you have not already seen:

1. The sender’s `kill(pid, sig)` asks the system server for the target’s signal channel, then `ConnectAttaches` to it.
2. It sends one pulse — `MsgSendPulse` with `code` set to the signal number and `value` to its own `pid` — and returns immediately. `kill` does not wait for the signal to be handled, exactly as POSIX specifies.
3. In the target, the system thread is sitting in `MsgReceive`. It wakes with the pulse, reads the signal number out of `code`, and runs the process-local delivery engine, which calls your handler.

So a signal handler runs **on the system thread**, in an ordinary thread context — not injected into the main thread between two instructions the way a classic Unix signal is. Skimmer has no `Signal*` system call at all; signals are built entirely from `libc` and pulses. (On the `seL4` variant the seam underneath differs, but the model above — and everything in the next section — is identical, because it lives in the shared C library, not the kernel.)

4.4 The POSIX signal surface

None of that changes how you *use* signals. The POSIX surface is the ordinary one and behaves as written:

- `signal` and `sigaction` install handlers;
- `kill` and `raise` send signals;
- `sigprocmask` and `pthread_sigmask` block and unblock them.

Two properties of the implementation are worth knowing, both invisible to correct POSIX code:

- **sigaction is process-local.** A signal disposition lives in your own process's C library; the system server never sees it. This matters only if you had assumed something about where dispositions are kept — the behavior you observe is the standard one.
- **Delivery is deferred, not asynchronous.** Because your handler runs on the system thread rather than interrupting the main thread mid-instruction, a whole category of async-signal-safety hazard simply does not arise. The trade is the ordinary one for any two threads: a handler and `main` share the address space, so data touched by both needs the synchronization of Chapter 5, not the old `volatile sig_atomic_t` dance.

Planned content. One piece of the surface is deliberately incomplete in 0.1: signaling a whole *process group* — `kill` with a non-positive pid — is not implemented and announces itself before returning `ENOSYS`. `kill` to a specific pid works fully. Code that relies on process-group signaling should treat it as a current gap, not a permanent one.

4.5 Pulses as notification: the `sigevent`

Signals are one fixed way to be notified; pulses are a more general one. The bridge between them — the way a program says *how* it wants to be told that something later happens — is the POSIX `struct sigevent`. A `sigevent` is a small descriptor of a notification-to-be:

- `SIGEV_PULSE` — deliver a pulse, on a given connection, with a chosen `code`, `value`, and priority. This is the native choice, and the QSOE header provides `SIGEV_PULSE_INIT` to fill one in.
- `SIGEV_SIGNAL` — raise a signal instead.
- `SIGEV_NONE` — record the event but deliver no notification.

You hand a `sigevent` to a facility that promises to notify you when its event occurs, and it later fires exactly the notification you described. The canonical example is a timer (Chapter 6): `timer_create` takes a `sigevent`, so you choose per timer whether its expiry arrives as a pulse or as a signal. The resource-server framework's readiness path (Chapter 8) is built on the same idea — a client arms the server with a `sigevent`, and the server delivers it when the resource becomes ready.

Planned content. The machinery that *delivers* an armed `sigevent` — `MsgDeliverEvent`, the server-to-client call that fires one — is **not yet implemented on either kernel**. The entry point exists but is an inert stub today; making it actually deliver an event is planned work for both QSOE/N and QSOE/L, along with the `poll()` readiness layer it feeds (the same

readiness gap the *Application Porting Guide* notes). So treat `sigevent` as the settled *shape* of asynchronous notification in QSOE — the descriptor you fill in and hand to a timer or a resource server — while knowing that the general delivery path behind it is still being built in 0.1.

Pulses, then, are not a niche corner of the system. They are how a process is told that time has passed, that a signal was sent, that a device it was waiting on is ready — the asynchronous counterpart to the synchronous rendezvous, and, as the next chapters show, the notification under the timer and the resource server alike.

Chapter 5

Synchronization

Two earlier chapters left the same debt. Chapter 2 said the threads of a process share its address space; Chapter 4 said a signal handler runs on the system thread, sharing memory with `main`. Both mean the same thing: when more than one thread touches the same data, the accesses must be coordinated. This chapter is the coordination toolkit — the native **Sync*** primitives and the POSIX threading calls that sit directly on them.

5.1 Two faces, one mechanism

QSOE does not have a native synchronization layer *and*, separately, a POSIX one. It has one mechanism with two faces. A `pthread_mutex_t` is a `sync_t` — the same storage under a different name — and `pthread_mutex_lock` is a one-line wrapper over `SyncMutexLock`. The same holds for condition variables and the rest. So the choice between them is only a matter of which spelling suits your code:

- **POSIX** (`pthread_mutex_lock`, `pthread_cond_wait`, ...) for portable code and anything that also builds elsewhere.
- **Native Sync*** when you want QSOE's own attributes directly — the mutex protocol, the process-shared flag — or when you are reading the system's servers, which use it.

The unit of storage is the `sync_t`: a small object you own, placed wherever you like (static, stack, or heap), and passed to every **Sync*** call *by address*. That address is more than a pointer to the state — it *is* the object's identity as a wait queue. There is no separate handle, no table in the system mapping ids to locks; the lock is the memory you put it in.

5.2 The primitives

One call creates any of the three kinds, selected by a type argument:

```
sync_t m;  
SyncTypeCreate(QSOE_SYNC_MUTEX_FREE, &m, NULL);    /* an unlocked mutex */
```

The types are `QSOE_SYNC_MUTEX_FREE` (a plain mutex, initially unlocked), `QSOE_SYNC_MUTEX_NONRECURSIVE` (which returns `EBUSY` rather than deadlocking on a nested lock), `QSOE_SYNC_COND` (a condition variable), and `QSOE_SYNC_SEM` (a counting semaphore). The operations are the expected ones:

- **Mutex** — `SyncMutexLock`, `SyncMutexUnlock`; and `SyncMutexRevive` to recover a mutex whose owner terminated while holding it (the robust-mutex case).

- **Condition variable** — `SyncCondvarWait(cond, mutex)` atomically releases the mutex and waits; `SyncCondvarSignal(cond, all)` wakes one waiter, or all of them when `all` is nonzero.
- **Semaphore** — `SyncSemPost` and `SyncSemWait(sem, try)`, where a nonzero `try` makes the wait non-blocking.

`SyncDestroy` releases an object when you are done with it. The canonical guarded-update shape — a mutex protecting state, a condvar to wait on a predicate — looks exactly as it does anywhere:

```
SyncMutexLock(&m);
while (!ready)           /* always loop on the predicate */
    SyncCondvarWait(&cv, &m); /* releases m, waits, re-acquires m */
consume(item);
SyncMutexUnlock(&m);
```

5.3 No futex: the address is the identity

QSOE has no futex layer, and the reason matters to how these calls perform. Because a `sync_t`'s address is its wait-queue identity, the **uncontended** path touches only that memory: locking a free mutex or unlocking one with no waiters is a single user-space atomic compare-and-swap on the `sync_t`, with no entry into the kernel or the system server at all.

Only *contention* costs more. When a thread must actually block — the mutex is held, the semaphore is empty — or a waiter must actually be woken, the operation reaches the system: on Skimmer through a kernel synchronization call, on the seL4 variant through an address-keyed wait/wake in the system server. Either way you pay only when a thread genuinely has to wait, never on the common fast path.

This is also why the `sync_t` differs in size between the two kernels (eight bytes on Skimmer, sixteen on seL4) while the API above does not differ at all: the storage layout is a per-kernel detail behind a seam header, and your code, using only the address and the calls, never sees it.

5.4 Priority inheritance: the real-time face

A mutex can carry a **protocol**, set through its `sync_attr_t` at creation — the analogue of POSIX's `PTHREAD_PRIO_*`:

- `QSOE_PRIO_NONE` — no priority adjustment.
- `QSOE_PRIO_INHERIT` — while a higher-priority thread is blocked on the mutex, its holder is boosted to that priority until it unlocks.
- `QSOE_PRIO_PROTECT` — the holder runs at the mutex's ceiling priority for as long as it holds it.

`QSOE_PRIO_INHERIT` is the answer to priority inversion in shared memory, the same hazard message passing handles for the request/reply path (§3.4). Without it, a low-priority thread holding a lock can stall a high-priority thread that needs it, for an unbounded time, while a middle-priority thread runs. With it, the holder is lifted to the waiter's priority and releases promptly. In a hard-real-time system the rule is simple: **any mutex a real-time thread can contend should use `QSOE_PRIO_INHERIT`**. Inversion is bounded the same way on both of QSOE's concurrency mechanisms — locks here, messages in Chapter 3.

5.5 Synchronizing across processes

A `sync_t` is private to one process by default. Created with `QSOE_SYNC_PROCESS_SHARED` in its `sync_attr_t` and placed in memory two processes have explicitly arranged to share, one object can coordinate threads in *different* processes.

This is the deliberate, asked-for sharing that Chapter 2 promised, never the accidental kind: it works only because you put the object in a shared region on purpose. It is also the less common answer. Between processes the QSOE-shaped tool is usually **message passing** (Chapter 3) — it carries priority inheritance for free and needs no shared memory at all. Reach for a process-shared `sync_t` in the specific case where two processes already share a memory region and must serialize access to its contents directly.

5.6 Choosing a tool

The three preceding chapters between them give a small, clean set of coordination tools; the choice among them is usually clear:

- **Within a process**, threads coordinate with this chapter’s mutexes, condition variables, and semaphores, in whichever face suits the code.
- **Between processes**, the default is message passing (Chapter 3): a request and reply, with inversion bounded automatically. Use a process-shared `sync_t` only for a genuinely shared memory region (§5.5).
- **For asynchronous events** — “something happened, do not wait” — the tool is a pulse or a signal (Chapter 4), not a lock.

Held to those lines, a QSOE program stays out of the corner where shared state and ad-hoc signaling tangle: data shared within a process is behind a lock, work shared between processes travels as messages, and one-way events travel as pulses.

Chapter 6

Time and Timers

A program needs two different things from a clock: to know what time it *is*, and to arrange to be woken *later*. QSOE keeps them separate, and keeps a third thing separate again — a periodic or asynchronous timer that *notifies* you on expiry, which is the time-flavored instance of the notification machinery of Chapter 4. This chapter takes the three in that order, and is candid about which are finished in 0.1 and which are still being built: reading time and sleeping for a duration work today; asynchronous timers are a settled shape with an unfinished delivery path.

6.1 Clocks: reading the time

Two clocks matter, named by a `clockid_t`:

- `CLOCK_MONOTONIC` — a steady count that only ever moves forward, never stepped or adjusted. This is the clock for measuring intervals and for deadlines: the right answer to “how long did that take” and “wake me 10 ms from now.”
- `CLOCK_REALTIME` — wall-clock / calendar time, the basis for timestamps a human reads.

The POSIX way to read either is `clock_gettime`; the native call is `ClockTime`. For the finest granularity, `ClockCycles` returns the raw RISC-V cycle counter (the `rdtime` register) directly, and the global `qsoe_time_freq_hz` gives its ticks-per-second so you can convert a cycle delta to nanoseconds. `ClockCycles` is the tool for timing a short, hot stretch of code where the overhead of a full `clock_gettime` would itself be part of what you are measuring.

Planned content. Reading the clocks works; *disciplining* the real-time clock does not yet. Setting it (`clock_settime`) and slewing it (`ClockAdjust`) are not wired in 0.1 — they report the honest POSIX failure (`EPERM` / `ENOSYS`) rather than silently dropping the change. A program that needs to read time is fully served; one that needs to *set* system time must wait.

6.2 Sleeping: blocking for a duration

To suspend a thread for a fixed span, use `nanosleep` or, to sleep until an *absolute* moment, `clock_nanosleep`. This is the timer facility that is solid today, on both kernels. It is pure time: the thread simply stops and resumes at the deadline, with no notification, no handler, no resource manager involved.

For anything periodic, sleep to an **absolute** deadline that you advance each cycle, rather than sleeping a relative interval — the latter accumulates the drift of whatever ran between wakeups, the former does not:

```

struct timespec next;
clock_gettime(CLOCK_MONOTONIC, &next);
for (;;) {
    next.tv_nsec += period_ns;
    while (next.tv_nsec >= 1000000000L) { /* normalize: 1e9 ns = 1 s */
        next.tv_nsec -= 1000000000L;
        next.tv_sec += 1;
    }
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL);
    do_periodic_work(); /* runs on a fixed cadence */
}

```

Underneath, `nanosleep` is the wired member of a native timed-blocking family (`TimerTimeout` with the `QSOE_TIMEOUT_NANOSLEEP` flag): block the calling thread, right here, until a deadline. That much is implemented and is what the code above rides.

6.3 Asynchronous and periodic timers

The third use of time is a timer that runs on its own and *notifies* you when it expires — the POSIX timer API. You create a timer against a clock, handing it a `sigevent` that says how you want expiry delivered, then arm it with an initial delay and (for a repeating timer) an interval:

```

timer_t t;
struct sigevent ev;
SIGEV_PULSE_INIT(&ev, coid, SIGEV_PULSE_PRIO_INHERIT, MY_TICK, 0);
timer_create(CLOCK_MONOTONIC, &ev, &t); /* expiry -> a pulse on coid */

struct itimerspec it = {
    .it_value = { .tv_sec = 0, .tv_nsec = period_ns }, /* first fire */
    .it_interval = { .tv_sec = 0, .tv_nsec = period_ns }, /* then every */
};
timer_settime(t, 0, &it, NULL);

```

This is where the `sigevent` of §4.5 pays off: a periodic timer armed with `SIGEV_PULSE` delivers a pulse to your channel every period, so a server’s ordinary receive loop (§4.2) gets a “tick” alongside its client requests — no special timing thread needed. Choosing `SIGEV_SIGNAL` instead would raise a signal on each expiry.

Planned content. The asynchronous side is the unfinished one in 0.1. The timer *objects* exist — `timer_create` / `TimerCreate` build and store a timer against its `sigevent` — but **arming and expiry delivery are not complete**. On the seL4 variant `timer_settime` (`TimerSettime`) is an announced `ENOSYS` stub today; and on both kernels the act of *delivering* the `sigevent` on expiry rides the same notification path Chapter 4 flagged as still being built. So write to this API and choose `SIGEV_PULSE` now — it is the settled shape — but do not yet rely on a periodic timer firing in 0.1. For a fixed delay or a periodic loop today, the absolute-deadline `clock_nanosleep` of the previous section is the facility that works.

6.4 Timed blocking

The native `TimerTimeout` call generalizes the deadline idea to the *other* blocking primitives: a `MsgReceive` that gives up after a timeout, a `SyncMutexLock` that will not wait forever, and so on — one flag per blockable state, in the shape QNX gives them.

Planned content. Of that family, only `QSOE_TIMEOUT_NANOSLEEP` — the engine behind §6.2 — is wired today. The timed send, receive, reply, mutex, condvar, semaphore, and join variants

are reserved and answer `ENOSYS` for now. The flag set is fixed, so code can be written against the shape; the implementations follow in later releases.

6.5 Time and real time

For the hard-real-time work QSOE is built for, the pieces above are the foundation rather than the whole story. A monotonic clock at `rdtime` resolution is what deadline scheduling is measured against, and the absolute-deadline `clock_nanosleep` is what gives a periodic real-time task its precise, drift-free cadence today — the worked loop of §6.2 is exactly the skeleton of a periodic RT thread. When asynchronous timer delivery lands, the same cadence becomes available as a pulse a server can fold into its receive loop. The scheduling guarantees that turn “wake me at this deadline” into “and run me *then*, ahead of lower-priority work” belong to the scheduler, and are the *Design & Architecture* guide’s subject; this chapter only provides the clock they are measured against.

Chapter 7

Interrupt Threads

Sooner or later a driver has to hear from hardware: a byte arrived, a transfer finished, a device wants attention. That is an interrupt, and the way a QSOE program handles one is the subject of this chapter — a way that will look slightly surprising if you have written interrupt handlers before, because on QSOE **there are no interrupt handlers**. There are only threads.

7.1 No handlers, only threads

The classic model is an interrupt service *routine*: a function you register with the kernel, which calls it — in a special, restricted context — each time the interrupt fires. That context is a minefield. The routine may not block, may not allocate, may not do most kinds of I/O or IPC; it runs outside the normal thread world, under rules all its own.

QSOE does not have this at all. There is no call that registers an interrupt *function*; the older QNX `InterruptAttach` that did is gone, and QSOE never carried it. Instead a driver dedicates a **thread** — an interrupt service thread, or IST — to the interrupt. The thread attaches itself to an interrupt source and then blocks, waiting. When the interrupt fires, the thread wakes and runs. That is the whole mechanism.

The gain is enormous and is the entire reason for the design: **an IST runs in ordinary thread context**. It is a normal thread that happens to wake on hardware. It may block, take a mutex, call `printf`, send a message to another server — anything any thread may do. None of the “what you must not do in an interrupt” rules apply, because it is not in an interrupt context; it is in a thread. A thread waking on an interrupt is the same scheduler entity as any other thread, with the same priority and the same standing — there is no second class of execution to reason about.

7.2 The interrupt API

Five calls make up the surface, and a driver typically uses the first three:

- `InterruptAttachThread(vector, flags)` — the calling thread becomes the IST for `vector`. Returns an interrupt id (`iid`). A thread attaches to at most one source.
- `InterruptWait(flags, timeout)` — block until the attached source next fires.
- `InterruptDetach(iid)` — give up the source.
- `InterruptMask / InterruptUnmask(vector, iid)` — explicitly hold off or re-enable the source, for the windows when you must (reconfiguring the device, say).

The `vector` follows one convention on both kernels: `QSOE_PLIC_VECTOR_BASE` plus the platform source number (the PLIC source, or the MSI identifier the PCI layer hands you). Using the same biased vector on Skimmer and on seL4 is deliberate — driver code computes it identically and ports between the kernels unchanged, while each libc seam maps it down to whatever its kernel actually exposes.

Two flags shape the common rhythm. Attaching with `QSOE_INTR_FLAGS_NO_UNMASK` leaves the source *masked* at attach time; the source then stays masked except while the thread is actually blocked in a wait, because `InterruptWait` is given `QSOE_INTR_WAIT_FLAGS_UNMASK`, which unmask-and-blocks as one atomic step. That pairing is what makes a single `InterruptWait` per fire correct: the source is open only during the wait, so an interrupt arriving while you are busy servicing the last one cannot be lost, and you never need a separate unmask call.

7.3 The canonical IST loop

Every interrupt thread in the system has the same shape: attach masked, then loop on wait-then-service. Stripped to its bones (this is the structure of the real serial, NVMe, and PCI drivers):

```
void *device_irq_thread(void *arg)
{
    int vec = QSOE_PLIC_VECTOR_BASE + device_source;
    int iid = InterruptAttachThread(vec, QSOE_INTR_FLAGS_NO_UNMASK);
    if (iid < 0) {
        /* announce the failure, then park -- never exit silently */
        fprintf(stderr, "irq attach failed: errno=%d\n", qsoe_errno);
        for (;;) { }
    }

    for (;;) {
        if (InterruptWait(QSOE_INTR_WAIT_FLAGS_UNMASK, NULL) < 0)
            continue;                /* re-arm and wait again */

        unsigned n = drain_device();    /* touch the hardware */
        if (n > 0)
            MsgSendPulse(server_coid, IST_PRIO, PULSE_DATA_READY, 0);
    }
}
```

Read it as three beats. *Attach* the source, masked. Then forever: *wait* (which unmask, blocks, and returns when the interrupt fires), *service* the device just enough to quiet it and pull its data, and *notify* whoever cares. The attach is checked and, on failure, the thread announces and parks rather than exiting — a silent missing IST would leave the device dead with no clue why.

7.4 The IST and the rest of the program

Notice what the loop's last line does, because it is the pattern that ties this chapter to the previous ones: the IST does *not* do the real work itself. It does the minimum the hardware demands — drain a FIFO, acknowledge the device — and then hands off to the server side of the program with a **pulse** (Chapter 4).

This is how a driver is shaped on QSOE. It is one process with two kinds of thread: a **main thread** that is a resource server, parked in `MsgReceive` answering client **reads** and **writes**, and **one IST per interrupt source**. The IST is the bridge from the hardware world into the message-passing world. When data arrives, the IST sends a pulse; the main thread, whose

receive loop already handles pulses alongside requests (§4.2), wakes and does the substantial work — including replying at last to any client it had parked with `MsgSavereply` (§3.6) waiting for exactly this data.

A pulse is the right tool for the handoff precisely because it does not block: the IST must return to its wait promptly so it is ready for the next interrupt, and a fire-and-forget pulse lets it. Were the IST to enter a synchronous `MsgSend` instead, it could block on the server and miss interrupts. So the division of labor is firm — the IST touches hardware and signals; the server thread talks to clients — and the pulse is the seam between them.

The real-time payoff follows from the IST being a true thread. Give it a high priority and it preempts lower-priority work the instant its interrupt fires, on any processor, the same as any high-priority thread (and recall from §3.7 that even a large in-kernel message copy yields to it at a page boundary). Nothing about handling an interrupt steps outside the scheduler’s ordinary, priority-driven world — which is exactly what a hard-real-time system needs, and exactly what the “no handlers, only threads” choice buys.

7.5 One source per thread

A thread attaches to a single interrupt source, by design. A device that raises several distinct interrupts gets several ISTs, one per source, each with its own attach-and-wait loop and its own priority. That keeps every loop as simple as the one above, and lets the scheduler weigh each source independently — a latency-critical completion interrupt can run at a higher priority than a housekeeping one on the same device. When the driver shuts a source down, the owning IST calls `InterruptDetach` and leaves its loop; until then, the masked-except-while-waiting discipline of §7.2 keeps each source quiet exactly when it should be.

Chapter 8

The Resource-Server Framework

Planned content. This chapter is in preparation.

Chapter 9

A Complete Resource Server

Planned content. This chapter is in preparation.

Appendix A

Native API Quick Reference

Planned content. This appendix is in preparation.